

Artificial Intelligence CS 407

Intelligent Agents

Part I: Agents

Throughout this course we shall ask one central question:

What does it mean for a system to behave intelligently?

Artificial Intelligence studies systems capable of

- perceiving their environment,
- reasoning about what they perceive,
- selecting appropriate actions, and
- learning from experience.

The central abstraction that unifies these ideas is the **intelligent agent**.

What is an Agent?

Russell and Norvig define an agent as

Definition

An **agent** is anything that

- 1 perceives its environment through sensors, and
- 2 acts upon that environment through actuators.

Examples include

- humans,
- autonomous robots,
- aircraft autopilots,
- web crawlers,
- recommendation systems,
- conversational AI assistants.

Agents Interact with their Environment



Perception and action form a continuous interaction loop.

A Mathematical Definition

Let P denote the set of all possible percepts. The agent observes a sequence

$$(p_1, p_2, \dots, p_t)$$

with $p_i \in P$, called the **percept history**.

Let A be the set of possible actions.

Agent Function

An agent is a function

$$f : P^* \rightarrow A$$

mapping every finite percept history to an action.

Example: Robot Vacuum Cleaner

Percepts:

$$P = \{\text{Dirty, Clean, Wall, Battery}\}$$

Actions:

$$A = \{\text{Forward, Turn Left, Turn Right, Suck, Recharge}\}$$

Example:

$$(\text{Clean, Wall, Dirty}) \mapsto \text{Turn Left}$$

The mapping is precisely an instance of

$$f : P^* \rightarrow A.$$

Agent Function versus Agent Program

These terms are often confused.

Agent Function

A mathematical specification

$$f : P^* \rightarrow A.$$

It tells us what the agent should do.

Agent Program

A computer program implementing the agent function on some computing architecture.

One agent function may have many different implementations.

Why is this Distinction Important?

Notice that the agent function makes no assumptions about

- logic,
- neural networks,
- machine learning,
- reinforcement learning,
- large language models.

Instead, it provides a universal abstraction capable of describing all of these systems.

Part II: Rational Agents

From Agents to Rational Agents

Not every agent behaves intelligently.

An intelligent agent should select actions that achieve its objectives.

Definition (Russell & Norvig)

A **rational agent** is one that selects the action expected to maximise its performance measure, given the percept sequence and its built-in knowledge.

Rationality depends upon

- what the agent has perceived,
- what it already knows,
- what actions are available.

A Mathematical View of Rationality

Let

$$h = (p_1, p_2, \dots, p_t)$$

denote the percept history.

Suppose

$$U : A \rightarrow \mathbb{R}$$

assigns a utility (or value) to every action.

A rational agent chooses

$$a^* = \arg \max_{a \in A} \mathbb{E}[U(a) \mid h].$$

The expectation reflects uncertainty about the consequences of our actions.

Performance Measures

A rational agent requires a criterion for success.

This criterion is called the **performance measure**.

Examples

Agent	Performance Measure
Vacuum Robot	Amount of dirt removed, energy consumed, time taken
Chess Program	Probability of winning
Taxi	Safety, travel time, fuel efficiency
Medical AI	Diagnostic accuracy, patient outcomes
LLM Assistant	Correctness, helpfulness, safety, latency

Rationality is not Omniscience

A common misconception is

“A rational agent always makes the correct decision.”

Rationality is not Omniscience

A common misconception is

“A rational agent always makes the correct decision.”

This is false.

A rational agent makes the **best possible decision** using the information currently available.

Examples

- Medical diagnosis
- Weather prediction
- Autonomous driving
- Financial trading

Perfect knowledge is rarely available.

Learning Improves Rationality

Experience changes behaviour.

Suppose the agent has experienced

$$h_1, h_2, \dots, h_t.$$

Learning modifies the agent function

$$f : P^* \rightarrow A$$

into a better approximation

$$f'.$$

Intelligence therefore consists not only of acting, but also of improving future decisions.

Task Environments

Agent behaviour depends strongly upon the environment.

Russell & Norvig classify environments according to several properties.

These properties determine the type of AI techniques that are likely to be successful.

Properties of Task Environments

Property	Possible Values
Observability	Fully / Partially Observable
Determinism	Deterministic / Stochastic
Episodes	Episodic / Sequential
Dynamics	Static / Dynamic
Representation	Discrete / Continuous
Agents	Single-Agent / Multi-Agent
Knowledge	Known / Unknown

Different combinations require different AI methods.

Examples of Task Environments

Task	Characteristics
Chess	Fully observable, deterministic, sequential, static, multi-agent
Robot Vacuum	Partially observable, dynamic, sequential
Medical Diagnosis	Partially observable, stochastic
Autonomous Driving	Partially observable, dynamic, continuous, multi-agent
ChatGPT	Partially observable, sequential, dynamic, language-based

To specify an intelligent agent, Russell & Norvig introduce the PEAS framework.

P	Performance Measure
E	Environment
A	Actuators
S	Sensors

PEAS provides a systematic method for describing AI systems.

Example: Robot Vacuum

Performance	Maximum cleanliness, minimum energy, minimum time
Environment	Rooms, walls, obstacles, charging station
Actuators	Move, turn, vacuum, dock
Sensors	Camera, lidar, bump sensor, battery sensor

Example: Conversational AI

Performance	Helpful, correct, safe, efficient responses
Environment	User, conversation, Internet, external tools
Actuators	Text, speech, tool calls, API requests
Sensors	User prompts, retrieval, memory, tool outputs

Notice that modern LLM-based systems fit naturally into the same PEAS framework.

Summary

In this section we introduced

- Rational agents
- Performance measures
- Expected utility
- Task environments
- PEAS

Next we ask

How can we build rational agents?

This leads naturally to **agent architectures**.

Part III: Building Rational Agents

Agent Architectures

An **agent architecture** specifies how the agent function

$$f : P^* \rightarrow A$$

is implemented.

Russell and Norvig describe five major architectures:

- 1 Simple Reflex Agents
- 2 Model-Based Reflex Agents
- 3 Goal-Based Agents
- 4 Utility-Based Agents
- 5 Learning Agents

As we move down this list, the agents become progressively more capable.

Simple Reflex Agents

Simple reflex agents respond only to the **current percept**.

$$a = f(p_t)$$

rather than

$$f(p_1, \dots, p_t).$$

Decision making is based on condition–action rules.

Examples

- IF dirty THEN vacuum
- IF obstacle THEN turn left
- IF battery low THEN recharge

Suitable only for fully observable environments.

Simple Reflex Agent



No memory.
No internal model.
No planning.

Model-Based Agents

Many environments are only partially observable.

The agent therefore maintains an internal model of the world.

Let

$$s_t$$

denote the internal state.

The state evolves according to

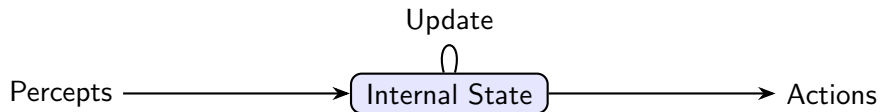
$$s_{t+1} = T(s_t, a_t, p_{t+1}),$$

where

- T is the state transition function,
- p_{t+1} is the new percept.

The action is chosen using

Model-Based Agent



The agent remembers aspects of the world that are not currently observable.

Goal-Based Agents

Instead of reacting immediately, the agent reasons about future states.

Let

$$G \subseteq S$$

be the set of goal states.

A planner computes a policy

$$\pi : S \rightarrow A$$

that moves the system towards G .

Typical applications

- Robot navigation
- Route planning
- Logistics

Planning

Planning searches for a sequence of actions

$$(a_1, a_2, \dots, a_n)$$

such that

$$s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n \in G.$$

Search algorithms studied later include

- Breadth-First Search
- Uniform Cost Search
- A*
- Heuristic Search

Planning therefore builds directly upon search.

Utility-Based Agents

Sometimes there are many goal states.
Some are better than others.

Define a utility function

$$U : S \rightarrow \mathbb{R}.$$

The rational action is

$$a^* = \arg \max_a \mathbb{E}[U].$$

Examples

- Fast route versus safest route
- Cheap versus comfortable travel
- Accuracy versus computation time

Goals versus Utilities

Goal-Based	Utility-Based
Reach a destination	Reach the best destination
Binary success/failure	Continuous preference
Planning	Decision making
Suitable for search	Suitable for optimisation

Utilities allow an agent to compare competing alternatives.

Learning Agents

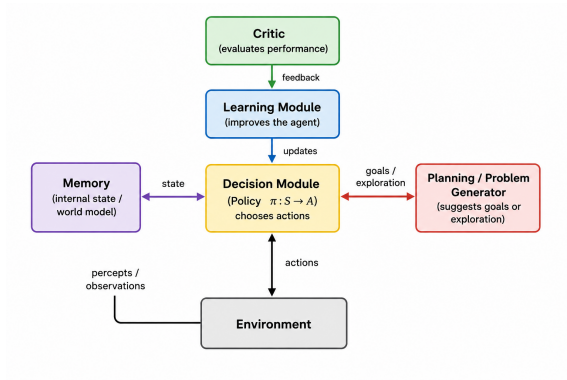
Learning agents improve their behaviour through experience.

Russell and Norvig identify four components.

- 1 Performance Element
- 2 Learning Element
- 3 Critic
- 4 Problem Generator

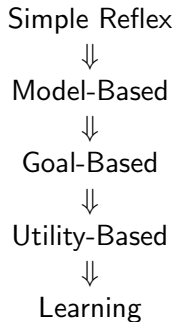
Together these continually improve the agent function.

Architecture of a Learning Agent



From Classical to Modern Agents

The five architectures form a natural progression.



Modern AI systems combine all five ideas.

Summary

In this section we studied

- Simple reflex agents
- Model-based agents
- Goal-based agents
- Utility-based agents
- Learning agents

These ideas have remained central to AI for over thirty years.

In the next lecture we shall see how modern **LLM-based agentic systems** extend these classical architectures.

Part IV: Modern Agents

From Intelligent Agents to “Agentic AI”

The idea of an **agent** is not new.

Recall the classical definition:

$$f : P^* \rightarrow A.$$

An agent maps a history of percepts to actions.

Modern “Agentic AI” asks:

Can foundation models be used to construct
more general-purpose agents?

The underlying scientific question remains the same.

Is an LLM an Agent?

Consider a standard Large Language Model:

$$x_1, x_2, \dots, x_t \longrightarrow x_{t+1}.$$

It computes a conditional distribution

$$P(x_{t+1} \mid x_1, \dots, x_t).$$

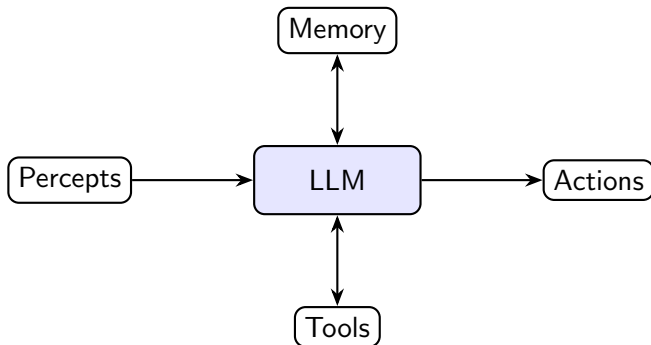
By itself, an LLM is primarily a **sequence prediction model**.

It does not necessarily have

- persistent goals,
- persistent memory,
- access to an external environment,
- the ability to take external actions.

From an LLM to an LLM Agent

We can place an LLM inside an agent architecture.



The LLM becomes one component of a larger agent system.

A Modern Agent Architecture

A modern LLM-based agent may contain

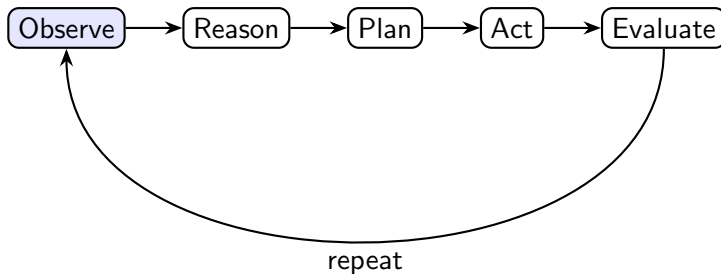
- a **foundation model**,
- a **working context**,
- **persistent memory**,
- explicit or implicit **goals**,
- a **planner**,
- external **tools**,
- mechanisms for evaluating outcomes.

Thus,

LLM Agent $\neq$ LLM alone

The Agentic Loop

A useful abstraction is



The agent repeatedly interacts with its environment rather than simply generating one response.

A Mathematical View

Let

$$o_t$$

be the observation at time t , and let

$$m_t$$

denote the agent's internal memory.

The agent computes

$$(a_t, m_{t+1}) = \pi(o_t, m_t, g),$$

where

- g represents its current goal,
- a_t is an action,
- m_{t+1} is the updated memory.

Memory

LLMs have a finite context.

An agent may therefore maintain additional state

$$m_t \in M.$$

Examples include

- conversation history,
- summaries of previous interactions,
- databases,
- vector stores,
- user-provided documents,
- records of previous actions.

Memory turns an otherwise transient interaction into a **stateful process**.

Tools

Actions need not be words.

Suppose the agent has a set of tools

$$\mathcal{T} = \{T_1, T_2, \dots, T_k\}.$$

The action space may contain

$$A = A_{\text{text}} \cup A_{\text{tool}}.$$

Tool actions might include

- searching the Web,
- querying a database,
- executing a program,
- using a calculator,
- retrieving a document,
- calling an external API.

Example: Tool-Using Agent

Suppose we ask:

“What is the cheapest flight to Delhi next Tuesday?”

The agent might

- 1 interpret the request,
- 2 determine what information is required,
- 3 call a flight-search tool,
- 4 inspect the returned results,
- 5 compare alternatives,
- 6 produce a recommendation.

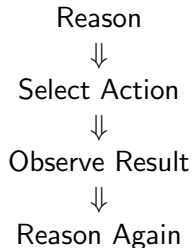
The LLM provides reasoning and language capabilities; the tools provide interaction with the external world.

Reasoning and Acting

One influential idea is to interleave

Reasoning and Acting.

For example,



This is closely related to the classical agent loop:

Perceive → Decide → Act.

Planning with LLM Agents

For a complex goal g , the agent may construct intermediate objectives

$$g \rightarrow (g_1, g_2, \dots, g_n).$$

For example:

“Organise a conference trip”

might require

- 1 find suitable flights,
- 2 find accommodation,
- 3 check conference dates,
- 4 estimate total cost,
- 5 construct an itinerary.

This is recognisably a form of classical AI planning.

Evaluation and Reflection

An agent can also evaluate the result of an action.

Let

$$r_t$$

represent feedback about the outcome.

The agent can use

$$(o_t, a_t, r_t)$$

to modify subsequent behaviour.

This resembles the **critic** in the Russell–Norvig learning-agent architecture.

Modern terms such as

reflection, self-critique, verification

can therefore be related to much older ideas in AI.

Classical Agents and Modern LLM Agents

Classical Agent Concept	Modern LLM-Agent Realisation
Percepts	Prompts, documents, images, tool outputs
Internal State	Context and persistent memory
World Model	Foundation model and external knowledge
Goals	User request or explicit task specification
Planning	LLM-generated plans and task decomposition
Actions	Responses and tool calls
Actuators	APIs, software tools, external services
Critic	Verification, feedback and reflection
Learning	Model training and adaptation mechanisms

Multi-Agent Systems

Instead of one agent, suppose there are

$$\mathcal{A} = \{A_1, A_2, \dots, A_n\}.$$

Agents may

- cooperate,
- compete,
- communicate,
- divide tasks,
- critique one another.

For example:

Planner $\rightarrow$ Researcher $\rightarrow$ Writer $\rightarrow$ Reviewer.

This connects modern LLM systems to the long-established field of **Multi-Agent Systems**.

What Makes Software “Agentic”?

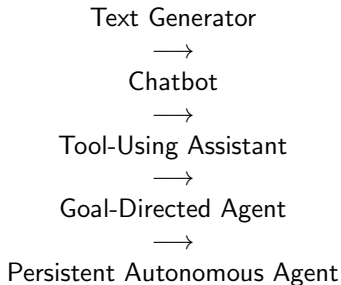
There is no single universally accepted definition.

However, agentic software typically exhibits several of the following:

- persistent interaction with an environment,
- goal-directed behaviour,
- maintenance of internal state,
- planning over multiple steps,
- autonomous selection of actions,
- tool use,
- evaluation of outcomes,
- adaptation of subsequent behaviour.

A Spectrum of Agency

It may therefore be better to think of agency as a spectrum.



Different systems exhibit different degrees of

- autonomy,
- persistence,
- environmental interaction.

Science versus Engineering

Recall the distinction from our first lecture.

AI Science	AI Engineering
Agency	LLM Agents
Reasoning	Foundation Models
Memory	Vector Databases
Planning	Agentic Workflows
Action	Tool Calling
Learning	Model Training
Multi-Agent Intelligence	Multi-Agent Frameworks

The Important Point

LLMs did not invent the idea of an agent.

The Important Point

LLMs did not invent the idea of an agent.

The scientific concept of an intelligent agent predates modern LLMs by decades.

The Important Point

LLMs did not invent the idea of an agent.

The scientific concept of an intelligent agent predates modern LLMs by decades.

LLMs provide a powerful new **engineering mechanism** for constructing agents.

The Agent as a Unifying Idea

The agent abstraction connects much of Artificial Intelligence.

Search	Finding actions
Logic	Reasoning about the world
Probability	Reasoning under uncertainty
Planning	Constructing sequences of actions
Learning	Improving from experience
Reinforcement Learning	Learning from rewards
Language	Communicating and reasoning
Multi-Agent AI	Interacting with other agents

These are different aspects of the **science of intelligent behaviour**.

Take-Home Messages

① An agent perceives an environment and acts upon it.

② The abstract agent function is

$$f : P^* \rightarrow A.$$

③ Rational agents select actions expected to maximise their performance.

④ Classical AI developed reflex, model-based, goal-based, utility-based and learning agents.

⑤ Modern LLM agents add powerful language models, memory, planning and tool use to this framework.